

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



INTELLIGENT SYSTEM

Recommendation System

Instructor: Quang Thanh Tho
Students: Luu Trong To - 2570802
Bui Ba Phuc - 2570799
Nguyen Ba Nam - 2570795

HO CHI MINH CITY, APRIL 2026

Contents

1	Introduction to Recommender Systems	1
1.1	Types of Recommendations	1
1.1.1	Non-Personalized Recommendations	1
1.1.2	Personalized Recommendations	2
1.2	Short Case Study: Netflix Recommendations	2
1.3	The Foundational Framework: The Utility Matrix	3
1.3.1	Representing the Utility Matrix	4
1.3.2	Data Collection Strategies	4
2	Stakeholders and Potential Benefits	5
2.1	End Users (Listeners)	5
2.2	Data Scientists	5
2.3	Project Managers	6
2.4	Summary	7
3	Theoretical Background	7
3.1	Content-Based Filtering with K-Nearest Neighbors (KNN)	7
3.1.1	Overview	7
3.1.2	K-Nearest Neighbors Algorithm	7
3.1.3	Application in This System	8
3.2	Collaborative Filtering (CF)	8
3.2.1	Overview	8
3.2.2	Memory-Based Collaborative Filtering	9
3.2.3	Item-Based Collaborative Filtering	9
3.2.4	Application in This System	9
3.3	Relationship Between the Two Approaches	10
4	Dataset Description and Preprocessing Steps	10
4.1	Music Information Dataset	10
4.2	User Listening History Dataset	11
4.3	Preprocessing Steps	11
4.3.1	Step 1: Character Encoding Correction	11
4.3.2	Step 2: Missing Value Handling	11
4.3.3	Step 3: Feature Selection for Content-Based Filtering	12
4.3.4	Step 4: Feature Normalization	12
4.3.5	Step 5: Referential Integrity Verification	12
4.3.6	Step 6: User-Item Matrix Construction	12
4.3.7	Step 7: Per-User Score Normalization	13
4.3.8	Step 8: Train-Test Splitting	13

1 Introduction to Recommender Systems

Recommender systems are software tools that help users discover items they are likely to find useful or interesting. In practice, they act as intelligent assistants that analyze user behavior, preferences, and available content in order to suggest relevant products, movies, music, videos, articles, or other digital items. Instead of forcing users to manually search through a huge collection of choices, a recommender system narrows the search space and presents a small set of meaningful options.

The need for recommender systems has become increasingly important in the digital era because modern online platforms contain an enormous number of items. E-commerce websites such as Amazon, streaming services such as Netflix, and content platforms such as YouTube provide users with millions of alternatives. Although such variety is beneficial, it also creates the problem of *information overload*. Users may spend too much time searching, may overlook suitable items, or may repeatedly select only the most obvious choices.

This challenge is closely related to the *Long Tail* phenomenon. In online environments, a small number of items receive most of the attention, while a very large number of niche items remain difficult to discover. A well-designed recommender system helps connect users with these less popular but highly relevant items, thereby improving user satisfaction and increasing the visibility of the full catalog.

Example: Long Tail Effect in Practice

A classic case is the book *Touching the Void*. Years after its original release, the book became a bestseller again because recommendation algorithms linked it to the newer and more popular title *Into Thin Air*. This example shows how recommender systems can revive interest in older or niche items by connecting them with popular products through user behavior and association patterns [2].

From the user perspective, recommender systems save time, reduce effort, and improve decision making. From the business perspective, they increase engagement, improve retention, support personalization, and can significantly boost sales or content consumption. For these reasons, recommender systems have become an essential component of many modern intelligent applications.

1.1 Types of Recommendations

Recommendation approaches can be divided into two broad categories depending on whether the suggestions are tailored to individual users.

1.1.1 Non-Personalized Recommendations

Non-personalized recommendation methods provide the same suggestions to every user without considering individual preferences, history, or behavior. Common examples include:

- **Editorial and hand-curated lists:** recommendations prepared by humans, such as “Top Picks” or “Editor’s Choice,” where experts decide which items should be highlighted.
- **Simple aggregates:** recommendations based on overall popularity, such as “Most Popular,” “Trending,” or “Top 10” lists.

These methods are easy to implement and useful for presenting broadly attractive items. However, they are limited because they do not reflect personal interests and therefore do not help users discover less well-known items that may better match their tastes.

1.1.2 Personalized Recommendations

Personalized recommendation systems are customized for each user. They analyze user behavior, such as what a person watches, clicks, rates, or buys, in order to suggest items that are more relevant to individual interests. Because these systems take personal preferences into account, they are generally more effective than non-personalized methods at helping users discover suitable content.

The main personalized recommendation approaches include:

- **Content-based systems:** these methods recommend items that are similar to those a user has liked before. For example, if a user often watches action movies, the system may suggest other action films with related themes, genres, or actors.
- **Collaborative filtering:** these methods recommend items based on the preferences of similar users. If users with tastes similar to a target user enjoyed a particular movie or product, the system may recommend that item as well.
- **Knowledge-based and hybrid systems:** knowledge-based methods use explicit information about user needs and item characteristics, which is useful in domains such as cars, houses, or financial products. Hybrid systems combine multiple recommendation techniques to improve accuracy and to address challenges such as limited data for new users or new items.

1.2 Short Case Study: Netflix Recommendations

Netflix provides a clear real-world example of how different recommendation strategies are used in the same platform.

Example: Non-Personalized Recommendations on Netflix

- **“Top 10 phim tại Việt Nam hôm nay”:** this row is driven by national viewing trends and regional popularity. It highlights what is widely watched in Vietnam rather than what matches one specific user’s preferences.
- **“Xem lại” (Continue Watching):** this feature follows a simple rule-based strategy. If a user starts a movie or series but does not finish it, Netflix shows it again so the user can continue watching.

Example: Personalized Recommendations on Netflix

- **“Danh sách của tôi” (My List):** this row contains movies and shows that the user has saved manually, so it directly reflects personal interests.
- **“Vì bạn đã xem...” (Because you watched...):** this type of row recommends titles related to the user’s previous viewing behavior.
- **“Đề xuất cho bạn” (Recommended for you):** this feature provides deeper personalization by combining viewing history with patterns learned from users who have similar preferences.

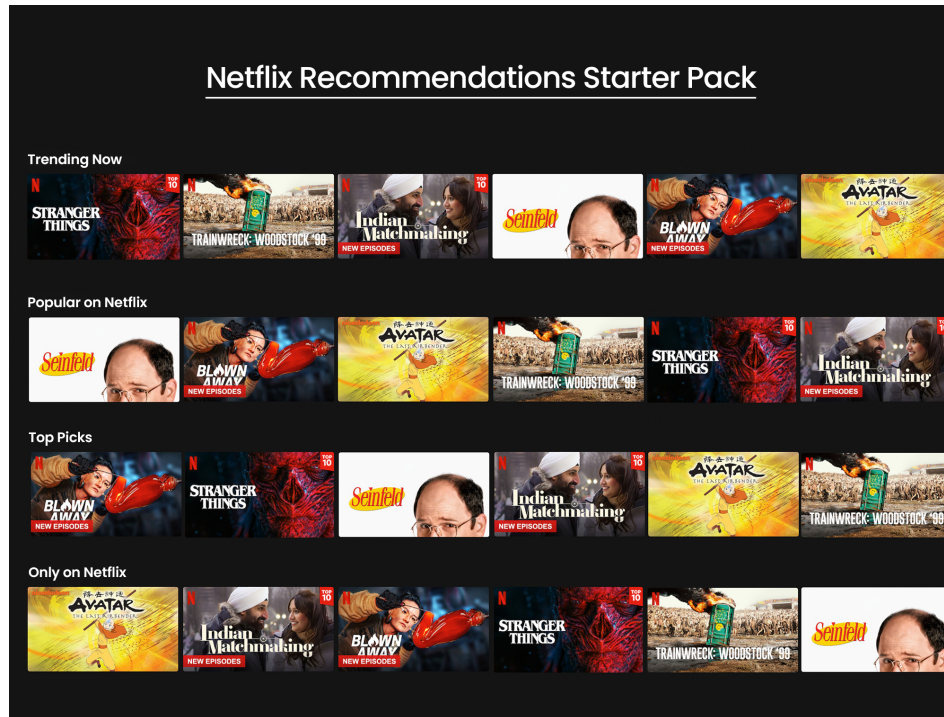


Figure 1: Netflix recommendation interface as an illustrative example.

1.3 The Foundational Framework: The Utility Matrix

The core of any recommendation system is the *utility matrix*, which records how useful or desirable each item is for each user. Formally, the recommendation task can be defined as a utility function:

$$u : X \times S \rightarrow R \quad (1)$$

where:

- X is the set of customers or users,
- S is the set of items such as products, news articles, movies, or videos,
- R is the set of ratings or utility values, for example a score from 1 to 5 or a normalized value in $[0, 1]$.

In practice, this function is represented as a matrix in which rows correspond to users and columns correspond to items. Each entry stores the observed or estimated preference of a user for a particular item. The central technical challenge in this framework is *sparsity*. In large-scale real systems, the utility matrix is overwhelmingly incomplete because most users interact with only a tiny fraction of the available catalog. As a result, most entries are unknown.

The goal of a recommender system is therefore not to fill the entire matrix perfectly. Instead, it aims to extrapolate the unknown entries that are likely to have high utility and then recommend

those items to the appropriate users. This idea is the foundation of both memory-based and model-based recommendation methods.

1.3.1 Representing the Utility Matrix

Table 1 shows a simple example of a utility matrix. Here, the symbol “?” represents unknown preferences that the system must estimate.

Table 1: A simple example of a utility matrix.

	Item A (Metallica)	Item B (Megadeth)	Item C (Into Thin Air)
User 1	5	?	1
User 2	?	4	?
User 3	4	5	?

This small example already illustrates the key problem. We know that User 1 strongly likes Item A but dislikes Item C. We also know that User 3 likes both Item A and Item B. Based on this limited evidence, a recommender system may infer that User 1 could also like Item B because Users 1 and 3 share a similar taste for heavy metal music. Likewise, the system may infer additional preferences for User 2 by comparing either similar users or similar items.

1.3.2 Data Collection Strategies

To build and update the utility matrix, recommendation systems rely on two main forms of feedback.

- **Explicit ratings:** users provide direct evaluations such as star ratings, likes, scores, or written reviews. These signals are usually high quality because they clearly reflect user intent. However, they suffer from data scarcity because many users do not want to spend extra effort rating items regularly.
- **Implicit ratings:** the system infers utility from user behavior, including clicks, purchases, watch time, dwell time, search activity, or repeated consumption. Although these signals are noisier than explicit ratings, they are extremely valuable in modern machine learning systems because they can be collected continuously at scale without increasing user friction. Netflix is a well-known example: in practice, recommendation quality depends heavily on behavioral signals such as what users watch, when they stop, and how they interact with the catalog, rather than on large volumes of explicit star ratings alone [1].

In real-world platforms, implicit feedback often becomes the main source of training data because it is abundant and naturally generated during normal use. For example, an online store may treat purchases as a strong positive signal, while a streaming platform may interpret finishing a movie or replaying a song as evidence of satisfaction.

Example: Netflix Beyond Explicit Ratings

Netflix illustrates why implicit feedback is so important in large-scale recommender systems. Besides explicit ratings, the platform can learn from many other signals, such as what users watch, when they watch, how long they keep watching, what device they use, what they search for, and which titles they add to their personal lists. Netflix also combines these user-side signals with item-side information such as popularity, metadata, and human-assigned

tags describing mood, style, or storyline. Together, these rich behavioral and content signals help the system understand user preferences much more effectively than explicit ratings alone [1].

To estimate the unknown entries caused by sparsity, recommender systems generally follow several major directions. They may analyze the *properties of items*, which leads to content-based reasoning, or analyze the *behavior of the community*, which leads to collaborative filtering. In addition, some systems rely on *knowledge-based recommendation*, where explicit information about user needs and item features is used to guide suggestions, especially in domains such as houses, cars, or financial products. Many practical platforms also adopt *hybrid approaches*, which combine content-based, collaborative, and knowledge-based signals in order to improve recommendation quality and reduce the weaknesses of any single method. Together, these perspectives form the conceptual bridge to the recommendation techniques discussed in the next sections.

In this report, however, **the main focus is on collaborative filtering**. The following discussion therefore concentrates on collaborative filtering methods, examples, and evaluation, while other recommendation paradigms are introduced only for context.

2 Stakeholders and Potential Benefits

This music recommendation system involves three primary stakeholder groups, each with distinct roles and benefits derived from the project.

2.1 End Users (Listeners)

End users are subscribed users who interact with the platform daily through two main surfaces: the **Homepage** and the **Player**. They have an existing listening history and provide both implicit feedback (play duration) and explicit feedback (like/dislike) to the system.

Potential Benefits:

- **Personalized Music Discovery.** The homepage recommendation surface delivers curated song suggestions tailored to each user's individual taste, enabling them to discover new music that aligns with their preferences without manual searching.
- **Seamless and Uninterrupted Listening Experience.** The next-song recommendation feature uses song-to-song similarity to suggest a contextually appropriate follow-up track, ensuring continuous playback and reducing drop-off between songs.
- **Increased Subscription Value.** By receiving a genuinely personalized experience, users gain greater satisfaction from their subscription, reinforcing long-term retention and platform loyalty.
- **User Control Through Feedback.** The like/dislike mechanism gives users direct influence over future recommendations, creating a personalized feedback loop that improves recommendation quality over time.

2.2 Data Scientists

Data scientists are responsible for designing, training, evaluating, and iterating on the machine learning models that power the recommendation engine. They operate at the core of the system's intelligence layer.

Potential Benefits:

- **Access to Rich and Structured Behavioral Data.** The system's event logging infrastructure captures fine-grained user interaction data — including play duration, play events, likes, and dislikes — providing both implicit and explicit signals necessary for building and validating recommendation models.
- **Support for Iterative Model Development.** The system architecture supports periodic model refresh and is designed with training and evaluation logging in mind, enabling data scientists to continuously experiment with new approaches and measure their real-world impact.
- **Modular and Scalable Pipeline.** The clear separation of components — candidate generation, ranking, and recommendation serving — allows data scientists to improve individual stages independently without disrupting the overall system, reducing iteration cost and deployment risk.
- **Measurable and Reproducible Evaluation.** Because all key interaction metrics are persisted in the backend database (PostgreSQL), data scientists can run offline evaluations and compare model versions against consistent historical data, supporting evidence-based decision-making.

2.3 Project Managers

Project Managers oversee the end-to-end delivery of the recommendation system, coordinating between engineering, data science, and product teams. They are responsible for defining success metrics, tracking business impact, and ensuring the system delivers measurable value aligned with organizational goals.

Potential Benefits:

- **CTR as a Primary KPI.** Click-Through Rate (CTR) — the ratio of users who click on a recommended song to the total number of recommendations displayed — serves as a direct, quantifiable measure of recommendation relevance. A higher CTR indicates that recommendations are resonating with users, giving Project Managers a clear signal to report to stakeholders. CTR is formally defined as:

$$\text{CTR} = \frac{\text{Number of Recommendation Clicks}}{\text{Total Recommendations Displayed}} \times 100\% \quad (2)$$

- **Data-Driven Decision Making.** With CTR tracked per recommendation surface (Homepage vs. Next-song), Project Managers can identify which surface delivers stronger engagement and prioritize engineering resources accordingly.
- **Iteration and Release Governance.** CTR trends across model versions provide objective criteria for approving or rolling back releases, reducing reliance on subjective judgment and accelerating the release cycle.
- **Business Impact Visibility.** An improvement in CTR directly correlates with increased listening time, lower churn, and higher subscription retention — metrics that Project Managers can translate into business value reports for leadership.
- **Balancing Quality vs. Quantity.** CTR also helps detect shallow optimization traps (e.g., clickbait recommendations with low completion rate). By pairing CTR with play duration data, Project Managers can ensure the system optimizes for genuine engagement rather than superficial clicks.

2.4 Summary

The three stakeholder groups are tightly coupled in a mutually reinforcing cycle. End users generate the behavioral and click data that data scientists rely on to improve models. Improved models in turn deliver better recommendations that increase user engagement and CTR. Project Managers monitor CTR as the shared bridge metric to govern releases, measure business impact, and align organizational expectations.

This feedback loop represents the core value proposition of the system — aligning the interests of all three stakeholder groups toward a shared goal of higher-quality, personalized music discovery.

Table 2: Stakeholder Summary

Stakeholder	Role	Primary Benefit
End Users (Listeners)	Consume recommendations	Personalized discovery & seamless playback
Data Scientists	Build & improve ML models	Rich data, modular pipeline, reproducible evaluation
Project Managers	Govern delivery & measure impact	CTR-driven KPIs, release governance, business visibility

3 Theoretical Background

This section presents the theoretical foundations underlying the music recommendation system, covering the two principal algorithmic paradigms employed: **Content-Based Filtering** and **Collaborative Filtering**.

3.1 Content-Based Filtering with K-Nearest Neighbors (KNN)

3.1.1 Overview

Content-based filtering recommends items to a user based on the **intrinsic attributes of the items** themselves, rather than the behavior of other users. The core assumption is that a user who enjoyed a particular item will likely enjoy other items with similar characteristics.

In the context of this music recommendation system, each song is represented as a **feature vector** composed of its metadata and audio attributes, including:

- Genre, subgenre, language, release year
- Precomputed audio features: danceability, energy, valence, tempo, acousticness, instrumentality, speechiness, and liveness

3.1.2 K-Nearest Neighbors Algorithm

KNN is a **non-parametric, instance-based learning** algorithm used to find the k most similar items to a given query item. Given a song s_q , the algorithm computes the distance between s_q and all other songs in the catalog, then returns the k songs with the smallest distance as candidates.

3.1.2.1 Distance and Similarity Metrics. The most commonly used metric is **Cosine Similarity**, which measures the angle between two feature vectors rather than their magnitude:

$$\text{cosine_sim}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (3)$$

Alternatively, **Euclidean Distance** can be applied when the magnitude of feature values is meaningful:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2} \quad (4)$$

3.1.3 Application in This System

Content-based KNN is primarily leveraged in the **Next-Song Recommendation** surface. Given the currently playing song, the system retrieves the k most similar songs based on their feature vectors. This approach operates entirely on song metadata, requiring no user history — making it robust for scenarios where interaction data is sparse.

3.1.3.1 Workflow:

1. Construct a feature vector for each song from its metadata and audio attributes.
2. Normalize the feature vectors to ensure each dimension contributes equally.
3. For a query song s_q , compute similarity scores against all catalog songs.
4. Return the top- k songs ranked by similarity as next-song candidates.

3.2 Collaborative Filtering (CF)

3.2.1 Overview

Collaborative Filtering recommends items by exploiting **patterns in user-item interactions**. Unlike content-based methods, CF does not rely on item attributes — it infers preferences purely from the collective behavior of users. The underlying assumption is: *users who agreed in the past tend to agree in the future*.

The primary input to a CF system is a **user-item interaction matrix** R , where each entry r_{ui} represents the interaction strength between user u and item i . In this system, the interaction signal is **play duration**, used as an implicit feedback proxy for preference intensity.

$$R = \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ r_{21} & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ r_{m1} & r_{m2} & \cdots & r_{mn} \end{pmatrix} \quad (5)$$

where m is the number of users and n is the number of songs.

3.2.2 Memory-Based Collaborative Filtering

Memory-based CF is a **lazy learning** approach that stores the entire interaction matrix and makes predictions at query time by directly computing similarities between users or items. It does not require a training phase and is straightforward to interpret.

There are two variants:

- **User-based CF:** Find users similar to the target user, then recommend songs those similar users have listened to.
- **Item-based CF:** Find songs similar to what the target user has interacted with, based on shared interaction patterns across users.

3.2.3 Item-Based Collaborative Filtering

Item-based CF is the chosen approach in this system because it produces more **stable similarity scores** — item relationships change less frequently than user behavior — and scales more efficiently when the user base is large.

3.2.3.1 Core Idea. Two songs are considered similar if they have been listened to by the same group of users in the same way.

3.2.3.2 Step 1 — Construct the item-item similarity matrix. For each pair of songs (i, j) , compute their similarity based on the co-interaction vectors extracted from R . The **Cosine Similarity** between song vectors $\vec{s}_i$ and $\vec{s}_j$ (each of length m , representing interactions from all users) is:

$$\text{sim}(i, j) = \frac{\vec{s}_i \cdot \vec{s}_j}{\|\vec{s}_i\| \cdot \|\vec{s}_j\|} \quad (6)$$

3.2.3.3 Step 2 — Generate candidates for a target user u . Let I_u be the set of songs user u has interacted with. For each candidate song $c \notin I_u$, estimate the preference score as a weighted sum:

$$\hat{r}_{uc} = \frac{\sum_{i \in I_u} \text{sim}(c, i) \cdot r_{ui}}{\sum_{i \in I_u} |\text{sim}(c, i)|} \quad (7)$$

3.2.3.4 Step 3 — Rank and return top- N recommendations. Songs are sorted by $\hat{r}_{uc}$ in descending order and the top- N are returned as recommendations.

3.2.4 Application in This System

Item-based CF is applied in the **Homepage Recommendation** surface to generate personalized candidate songs for each subscribed user. The system uses **play duration** as the implicit interaction signal r_{ui} , capturing listening depth as a proxy for user preference. Explicit signals (like/dislike) can further refine these scores as supplementary weights.

3.3 Relationship Between the Two Approaches

The two methods are complementary and are combined within the system's candidate generation pipeline, as summarized in Table 3.

Table 3: Comparison of Content-Based KNN and Item-Based CF

Dimension	Content-Based (KNN)	Item-Based CF
Data source	Song metadata & audio features	User-item interaction history
Similarity basis	Feature space proximity	Co-interaction patterns
Primary surface	Next-song recommendation	Homepage recommendation
Cold-start handling	Handles item cold-start well	Requires interaction history
Personalization level	Item-centric	User-behavior-driven

Together, these approaches form a **hybrid candidate generation layer**: content-based KNN supplies song-to-song similarity candidates for continuous playback, while item-based CF supplies personalized candidates derived from the broader listening community's collective behavior.

4 Dataset Description and Preprocessing Steps

This section provides a detailed description of the two datasets used to design and evaluate the music recommendation system: the Music Information dataset and the User Listening History dataset. It further documents the full sequence of preprocessing steps applied to both datasets before model training and evaluation.

4.1 Music Information Dataset

The Music Information dataset serves as the item catalog for the content-based filtering component of the recommendation system. It contains 1,500 song records, each uniquely identified by a `track_id` derived from the MusicBrainz database, with no duplicate entries across the catalog. The dataset spans 58 distinct release years ranging from 1933 to 2019, providing a broad historical coverage of recorded music across multiple decades and genres.

Each record contains 18 fields organized into three distinct categories. The first category covers track identifiers and metadata: `track_id` (unique MusicBrainz identifier), name (song title), artist, year of release, `spotify_id`, `spotify_preview_url`, and tags (a free-text field containing user-assigned genre and mood descriptors). The second category is a categorical genre label: each song is assigned to exactly one of 15 genres — Rock, Electronic, Metal, Jazz, Country, Rap, Reggae, Blues, Pop, RnB, Punk, Folk, Latin, New Age, and World — with a perfectly balanced distribution of 100 songs per genre. This balanced representation eliminates genre-level class imbalance and supports fair cross-genre evaluation. The third category comprises nine numerical audio feature attributes computed by the Spotify audio analysis engine: danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, and tempo. These nine features form the primary feature vectors used by the K-Nearest Neighbors algorithm for content-based similarity computation.

The audio features exhibit wide value ranges that reflect the natural diversity of musical styles in the catalog. Danceability ranges from 0.060 to 0.980, energy from 0.000 to 0.999, valence from

0.022 to 0.983, and tempo from 36.849 BPM to 219.993 BPM. Loudness, expressed in decibels, ranges from -32.779 dB to -0.590 dB, consistent with standard audio mastering conventions in which higher (less negative) values indicate louder average signal levels. Across all 18 columns, the only field containing missing values is tags, which has 81 null entries (approximately 5.4% of records). All other columns — including every audio feature — are fully populated with no missing values.

4.2 User Listening History Dataset

The User Listening History dataset captures the interaction signals between users and tracks, forming the foundation of the collaborative filtering component. It contains 45,891 interaction records linking 30,089 unique users to 895 unique tracks. A cross-validation check confirms that every `track_id` present in this dataset has a matching entry in the Music Information catalog, establishing complete referential integrity between the two sources. All six columns are fully populated with no missing values in any field.

Each record contains six fields. The `track_id` and `user_id` fields identify the song and the listener, respectively. User identifiers are stored as SHA-1 hashed strings to preserve user anonymity, ensuring that no personally identifiable information is retained in the dataset. The `playcount` field records how many times a given user has played a specific track, with values ranging from 1.01 to 499.99 and a mean of approximately 249.21. The near-uniform spread of playcount values across the full range suggests these figures may have been normalized or synthetically scaled during dataset preparation. The `is_like` and `is_dislike` fields capture explicit binary feedback signals, where a value of 1 indicates a positive or negative user reaction, respectively. Approximately 49.7% of interaction records carry a like signal and 25.4% carry a dislike signal; both signals may co-occur or be simultaneously absent within a single record. Finally, the `score` field provides a composite engagement metric that combines playcount magnitude with explicit feedback polarity. Score values range from -0.598 to 100.798 with a mean of approximately 50.04, indicating an approximately linear relationship to playcount with upward or downward adjustments applied for like and dislike signals.

4.3 Preprocessing Steps

Before any model is trained or evaluated, both datasets undergo a sequence of preprocessing steps to ensure data quality, structural consistency, and compatibility with the algorithms employed. Each step is described below.

4.3.1 Step 1: Character Encoding Correction

Both CSV files are loaded using Latin-1 (ISO 8859-1) encoding rather than the default UTF-8. This is necessary because the artist name and tags fields contain accented and special characters from non-English-language music (e.g., French, Spanish, and Portuguese artist names) that cannot be decoded correctly under UTF-8 without raising a parsing error. Applying Latin-1 encoding at the point of ingestion prevents character corruption in song titles, artist names, and tag strings throughout the entire pipeline.

4.3.2 Step 2: Missing Value Handling

The only column with missing data is the tags field in the Music Information dataset, which contains 81 null entries (5.4% of records). Because tags are used only as supplementary descriptive metadata and are not part of the numerical audio feature vector, missing entries are filled with an

empty string. This allows downstream text-processing operations to execute without raising null-reference errors while preserving the integrity of the feature matrix. No imputation of numerical values is required, as all nine audio feature columns and all six columns of the User Listening History dataset are fully complete.

4.3.3 Step 3: Feature Selection for Content-Based Filtering

For the KNN-based content-based model, the feature vector for each song is constructed exclusively from the nine Spotify audio attributes: danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, and tempo. Metadata fields such as `track_id`, name, artist, `spotify_preview_url`, `spotify_id`, and tags are excluded from the numerical matrix because they are either non-comparable identifiers or unstructured text. The genre and year fields are retained separately and used as categorical filters during candidate retrieval, rather than as dimensions in the distance computation.

4.3.4 Step 4: Feature Normalization

The nine audio features operate on heterogeneous numerical scales that would otherwise distort distance-based similarity computations. Tempo spans a range of approximately 36.849 to 219.993 BPM, whereas most other features such as danceability, energy, and valence are bounded within $[0, 1]$. Loudness, measured in decibels, takes strictly negative values between -32.779 and -0.590 . To prevent high-magnitude features from disproportionately dominating cosine similarity calculations in KNN, all nine audio features are rescaled to the $[0, 1]$ range using min-max normalization. After normalization, each feature dimension contributes proportionally to the overall similarity score, ensuring that no single attribute biases the neighbor retrieval process due to its original scale.

4.3.5 Step 5: Referential Integrity Verification

Before constructing any model, the `track_id` keys present in the User Listening History dataset are cross-validated against the Music Information catalog. All 895 tracks in the listening history are confirmed to have matching entries in the music catalog, establishing full referential integrity. This check prevents silent lookup failures in which a track recommended by the collaborative filtering model cannot retrieve its audio feature vector because it is absent from the content catalog. Tracks that appear in the music catalog but have no listening history entries are retained for content-based recommendation but excluded from the collaborative filtering model, as no interaction signal is available for them.

4.3.6 Step 6: User-Item Matrix Construction

For the collaborative filtering component, the User Listening History dataset is pivoted into a user-item interaction matrix in which rows represent users, columns represent tracks, and each populated cell contains the `score` value as the preference signal. The resulting matrix has dimensions of 30,089 users by 895 tracks. Since only 45,891 of the possible 26,929,655 user-track pairs are observed, the matrix density is approximately 0.17%, corresponding to a sparsity of 99.83%. This level of sparsity is typical of real-world implicit-feedback datasets. Unobserved cells are treated as missing rather than zero, to correctly distinguish between a user who has not yet encountered a track and a user who has actively disliked it.

4.3.7 Step 7: Per-User Score Normalization

The `score` column, ranging from -0.598 to 100.798 , serves as the preference signal for collaborative filtering. Because users differ significantly in their listening activity levels — some users have interacted with many tracks and accumulated large absolute scores, while others have minimal history — raw scores cannot be compared fairly across users. To address this, scores within the user-item matrix are normalized on a per-user basis using min-max scaling, mapping each user's preference profile onto the $[0, 1]$ interval. This per-user normalization reduces the dominance of highly active users in similarity computations and ensures that less active users contribute equally informative preference signals to the model.

4.3.8 Step 8: Train-Test Splitting

For offline evaluation of the collaborative filtering model, interaction records are partitioned into training and test sets at the user level. For each user who has recorded at least two distinct interactions, a held-out subset of their interactions is designated as the test set, while the remaining interactions form the training set. This user-level splitting strategy guarantees that every evaluated user has an observed history in the training data, which is a prerequisite for generating meaningful personalized recommendations. Users with fewer than two interactions are excluded from the evaluation partition but may still receive recommendations through the content-based fallback mechanism, which requires no prior interaction history to produce results.

References

- [1] Xavier Amatriain, Justin Basilico, and Jisoo Joo. Recommender systems in industry: A netflix case study. In *Recommender Systems Handbook*, chapter 8.1: Beyond explicit ratings. 2015. <https://amatria.in/pubs/Recsys-in-industry.pdf>.
- [2] Dinesh Asanka. Into thin air to touching the void. <https://dbfriend.blogspot.com/2021/02/into-thin-air-to-touching-void.html>. Data is everywhere, but?, 27/02/2021.